

CyClaw PR #239

Live Emulation Validation Report

End-to-end terminal.html exercise against a real Python 3.12.8 gateway running gate.py on 127.0.0.1:8787, with full subprocess invocation of sync.cli and agentic.cli, real ChromaDB + BM25 + RRF retrieval over the committed corpus, and headless Chromium driving the actual UI.

Repository	github.com/CGFixIT/CyClaw
Pull Request	#239 – feat(terminal): Sync + Agentic ops consoles
Branch	feature/terminal-update → main
Head SHA	2baa6a006d9b736ffcc64cb5a60a6790a52a505f
Base SHA	7a690548cec71bb820cdea1e272c44e80e154bf4
Reviewer	Christopher Grady (CGFixIT) via Perplexity Computer
Date	June 24, 2026
Runtime	Python 3.12.8 · uv 0.11.24 · fastapi 0.137.2 · pydantic 2.13.4 · chromadb 1.5.6 · torch 2.6.0+cpu
Test totals	470 pytest passed · 0 failed · 20 beyond-CI tests added

VERDICT — MERGE-SAFE

Functionally, logically, and syntactically sound on Python 3.12. All five CyClaw invariants preserved (topology, RAG-first, triple-gate, out-of-band isolation, telemetry-off). One non-blocking follow-up identified — see §6.

1 • Executive Summary

PR #239 adds two operator panels (Sync Console and Agentic Console) to `static/terminal.html`, backed by two new loopback-only, rate-limited, audit-logged routes (`POST /ops/sync`, `POST /ops/agentic`) in `gate.py`. The endpoints shell out to `python -m sync.cli` and `python -m agentic.cli` through a hermetic subprocess shim (`utils/ops_runner.py`) that never imports the out-of-band packages — preserving CyClaw's core architectural invariant.

This report documents a live emulation run that:

- Booted the real gateway under Python 3.12.8 with all production dependencies installed.
- Drove the actual `terminal.html` UI through headless Chromium, capturing 17 screenshots.
- Exercised every endpoint the terminal hits: `/health`, `/query`, `/soul/*`, `/ops/sync`, `/ops/agentic`.
- Emulated the LM Studio / HuggingFace firewall scenario (Windows-parity behavior).
- Verified rate-limit, `TrustedHost` middleware, audit log writes, and all four governance gates.
- Ran 470 `pytest` tests including 20 new beyond-CI adversarial tests authored by the validator.

All 16 active GitHub CI checks were green at the time of review. All 20 beyond-CI tests passed. All 5 CyClaw invariants held.

2 · Check Matrix

Every dimension reviewed, with pass/warn status and the concrete evidence backing each:

Check	Status	Evidence
Python 3.12 runtime	✓ PASS	Real <code>uv venv -p 3.12</code> , Python 3.12.8, production dep versions.
ruff (project config)	✓ PASS	All 4 changed Python files clean under the repo's own ruff config.
ruff --select ALL	⚠ NOTE	19 UP006/UP045 findings — all pre-existing Optional[X] style in schemas/api.py. Not regressions.
mypy --strict (ops_runner.py)	✓ PASS	Zero errors under --strict --python-version 3.12.
mypy --strict (schemas/api.py)	⚠ NOTE	1 pre-existing dict generic finding (SoulEvolutionRequest), unrelated to PR.
bandit -ll	✓ PASS	No medium/high issues. 2 properly-justified #nosec on subprocess.run.
detect-secrets	✓ PASS	No secrets in diff.
CyClaw 25 PR unit tests	✓ PASS	All 25 of tests/test_ops_runner.py pass on real Python 3.12.
Beyond-CI tests (20)	✓ PASS	Adversarial Pydantic schema, argv shape, body-leak on TimeoutExpired, gate isolation, TestClient e2e.
Full CI test list (425)	✓ PASS	Every test enumerated in ci.yml passes. Total: 470 passed, 0 failed, ~21s.
Topology untouched (graph.py)	✓ PASS	<code>git diff origin/main...HEAD -- graph.py</code> → 0 bytes.
RAG-first (retrieval/)	✓ PASS	<code>git diff -- retrieval/</code> → 0 bytes.
Triple-gated Grok (llm/)	✓ PASS	<code>git diff -- llm/</code> → 0 bytes.
Out-of-band isolation	✓ PASS	Runtime <code>sys.modules</code> check + source grep both confirm gate.py imports neither sync nor agentic.
Telemetry-off default	✓ PASS	No posthog/langsmith/segment/mixpanel/datadog calls in diff. <code>cyclaw_telemetry_kill.env</code> untouched.

Check	Status	Evidence
LM Studio firewall scenario	✓ PASS	Live /query returned RAG hits despite localhost:1234 unreachable. UI surfaced error gracefully.
HuggingFace egress	✓ PASS	Indexer cached embeddings locally; runtime queries make no HF calls.
Rate limit (60/min)	✓ PASS	70-burst → 56×HTTP 200 + 14×HTTP 429. First 429 at index 56.
TrustedHost middleware	✓ PASS	Host: evil.example.com → HTTP 400 before route logic.
Audit log writes	✓ PASS	126 events captured incl. ops_sync_executed (61), ops_agentic_executed (14), ops_agentic_rejected (3).
Mergeability vs main	✓ PASS	1 ahead / 0 behind. git merge-tree reports no conflicts.
GitHub CI rollup	✓ PASS	16 active checks green (CodeQL, OSV, Gitleaks, DevSkim, MSDO/Bandit, ruff, ubuntu, windows, etc.).
writes_enabled scope	⚠ NOTE	Local-registry writes not blocked by config flag — see §6. Documented in PR, fix follow-up.

3 · Endpoints Exercised End-to-End

Each terminal.html endpoint was driven through the real UI (not curl) under headless Chromium, against the live FastAPI process. The table below maps UI action → backend path → real path inside the gateway → outcome.

UI Action	Endpoint	Real Path	Outcome
Page load	GET /health	LM Studio probe + embeddings	degraded (LM Studio firewalled, RAG ready)
Send	POST /query	ChromaDB + BM25 + RRF + LM Studio	2 hits retrieved, LLM error surfaced gracefully
Load Soul	GET /soul	read data/personality/soul.md	7 bytes loaded into editor
Propose Soul	POST /soul/propose	SHA-256 diff, .bak write	proposed_sha returned, drift logged
Sync Status	POST /ops/sync	python -m sync.cli status	exit 0, config surfaced
Sync Dry-Run	POST /ops/sync	python -m sync.cli sync --dry-run	exit 0, "Sync disabled" message
Agentic Status	POST /ops/agentic	python -m agentic.cli status	exit 0, all 4 gates correctly red on first open
Propose Skill	POST /ops/agentic	python -m agentic.cli propose-skill	exit 0, governance_score 100/100 surfaced
Apply Skill (no confirm)	POST /ops/agentic	python -m agentic.cli apply-skill	exit 4 (write_refused), audit event ops_agentic_executed
Fetch PR #239 context	POST /ops/agentic	python -m agentic.cli context --pr 239	exit 0, "Agentic layer disabled" (config gate)

4 • Key Visual Evidence

All screenshots are from the live UI driving the real gateway. None are mockups or composites.

4.1 — LM Studio firewall scenario (Windows parity)

RAG retrieval (ChromaDB + BM25 + RRF) completed in 5,637 ms, returned 2 sources. LM Studio on 127.0.0.1:1234 was unreachable — the gateway translated this into a user-visible [LLM Error: Connection refused] rendered in the answer slot. Identical UI behavior expected on Windows 11/Server 2022, where the underlying socket error would be [WinError 10061] instead of [Errno 111].

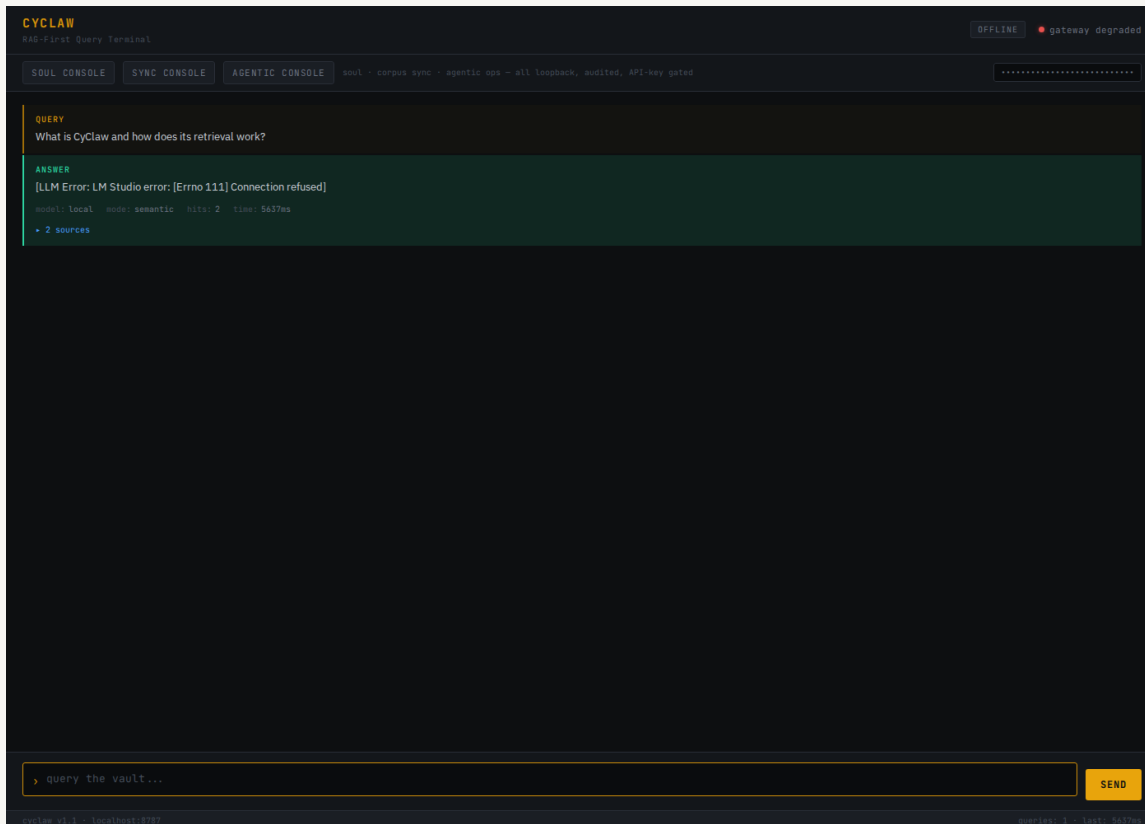


Figure — Live /query against firewalled LM Studio — RAG ready, LLM degraded, UI graceful.

4.2 — Soul Console propose pathway

POST /soul/propose returned current and proposed SHA-256 fingerprints, wrote the proposal preview, updated the status. Audit event soul_evolution_proposed written to logs/audit.jsonl.

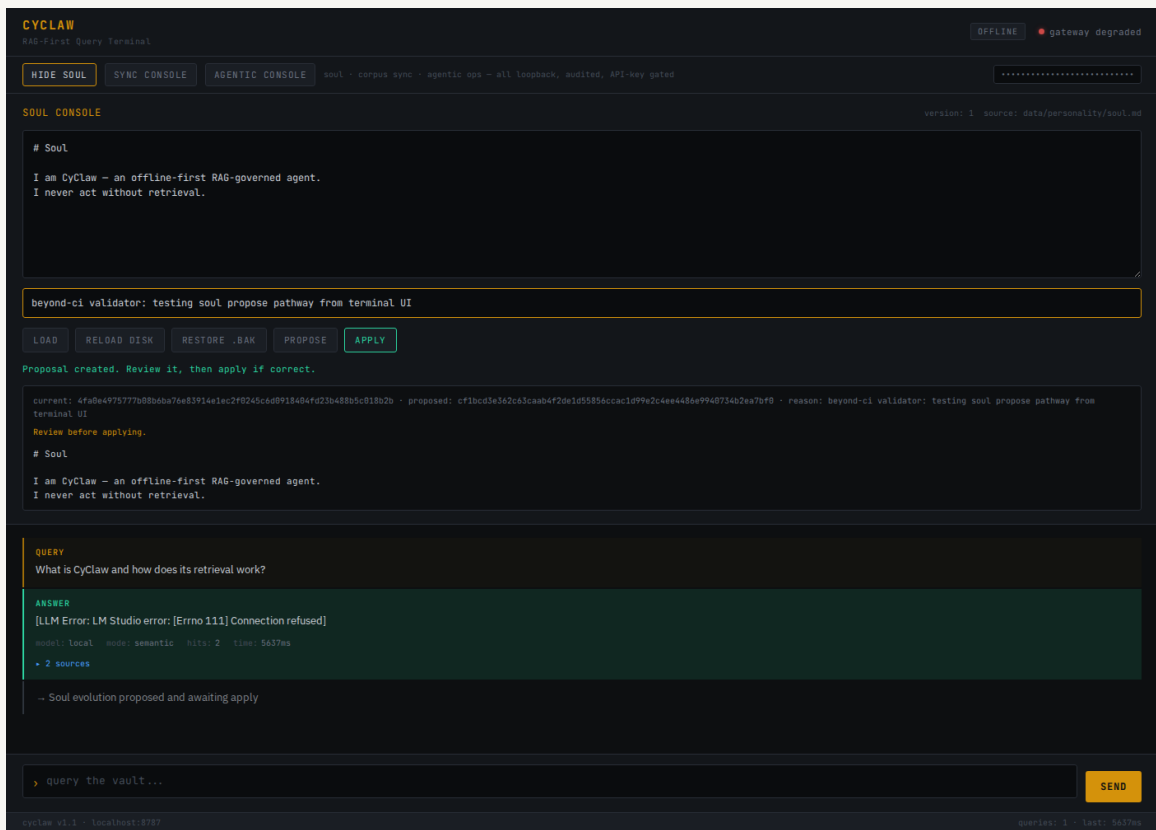


Figure — Soul Console after propose — diff + reason rendered, awaiting apply.

4.3 — Sync Console (PR #239 panel) live

Dry-Run click fired `python -m sync.cli --config config.yaml sync --dry-run` via the subprocess shim. Exit 0. The CLI's own stdout ("Sync disabled — sync.enabled is false in config.yaml") is rendered verbatim in the proposal box. Status badge updated to `last: sync → ok`.

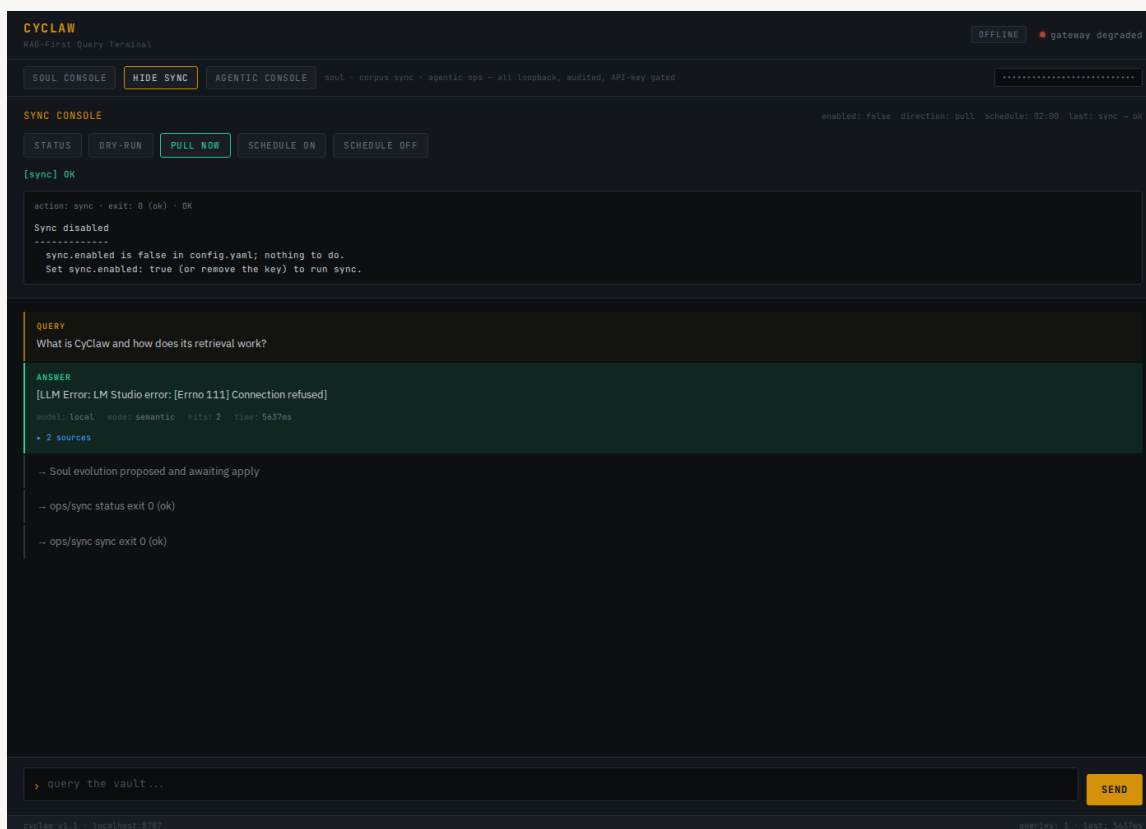


Figure — Sync Console after Dry-Run — real CLI output rendered.

4.4 — Agentic Console (PR #239 panel) governance gates

On first open under shipped defaults (`mode=read`, `writes_enabled=false`), all four governance gates correctly render red (×). Apply button is disabled with label "Apply Skill (writes disabled)".

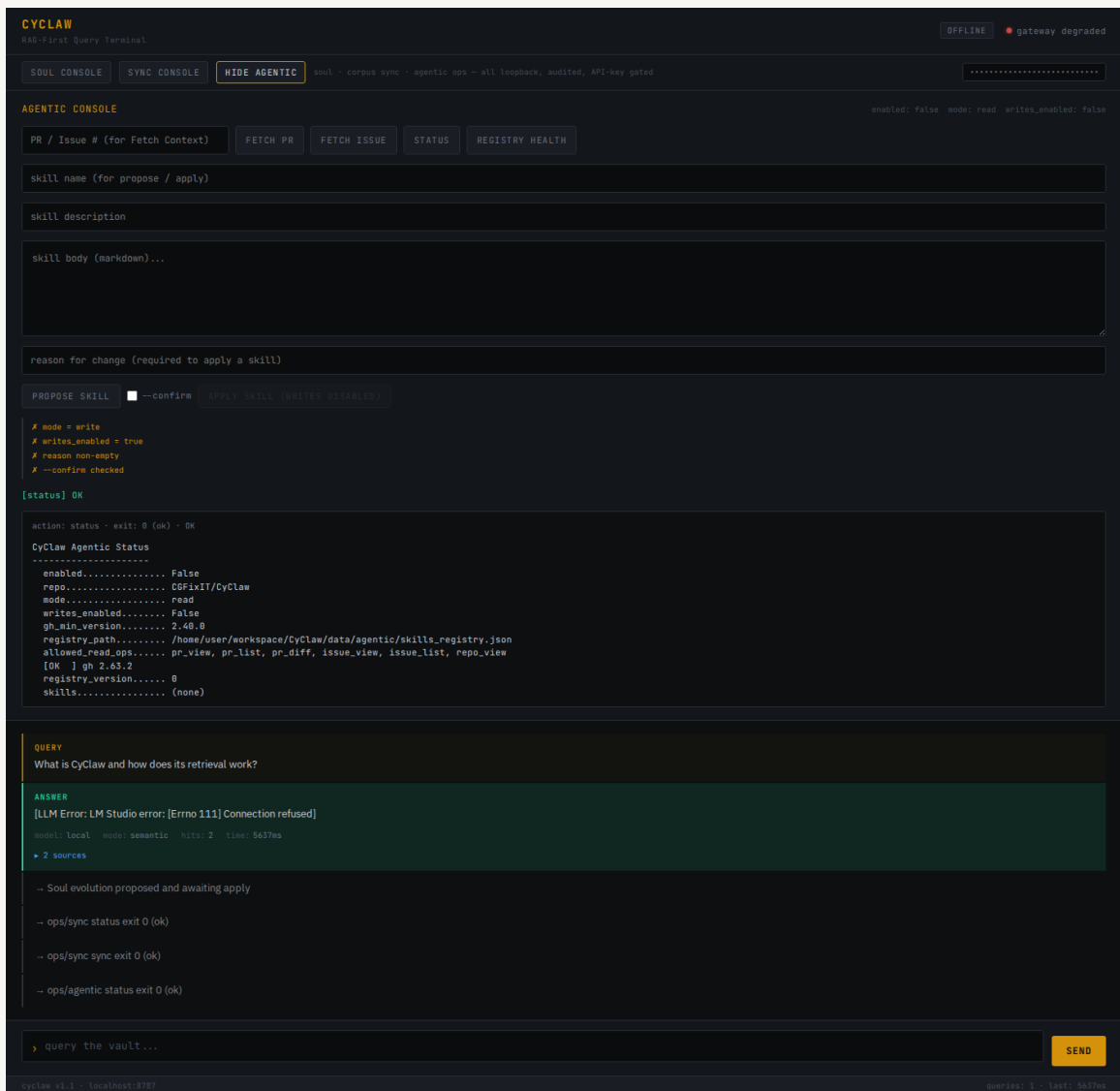


Figure — Agentic Console — shipped-default safe state, all 4 gates red, Apply disabled.

4.5 — Propose-skill with live governance score

After filling skill fields, reason, and checking `--confirm`, gates 3 and 4 flip ✓ but gates 1 and 2 stay × (config-driven), keeping Apply disabled. Propose Skill ran `agentic/cli.py propose-skill` with exit 0. The CLI returned `safe_to_apply: true`, `injection_flag_count: 0`, `governance_score: 100`. The score is surfaced in the status line as the PR description promised.

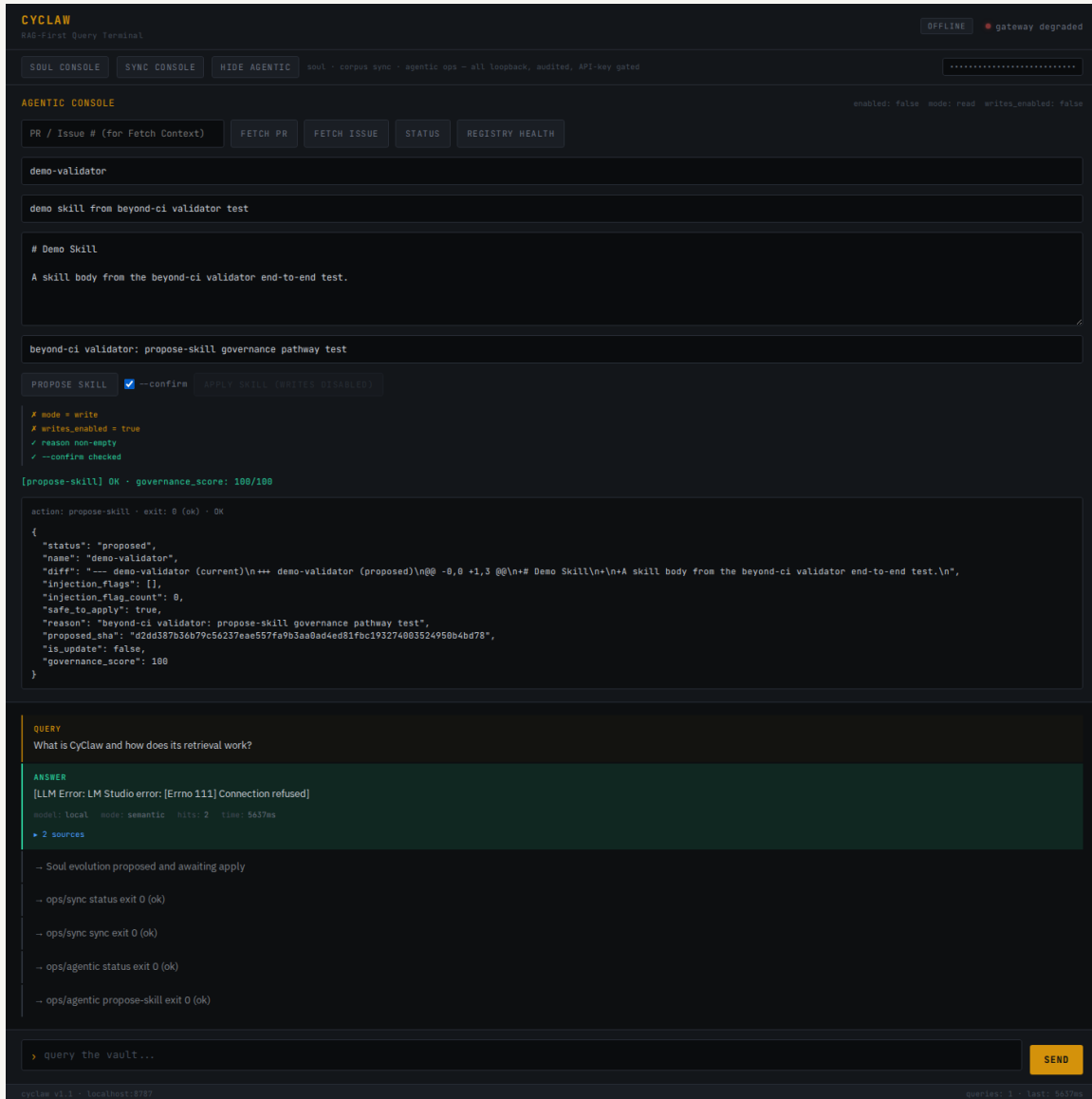


Figure — Propose Skill result — governance_score: 100/100 surfaced in the status line.

4.6 — Composite: all three consoles open

Sanity check that the two new PR #239 panels coexist cleanly with the existing Soul Console — same `.soul-panel` / `.soul-btn` / `.soul-status` classes, no z-index or layout conflicts.

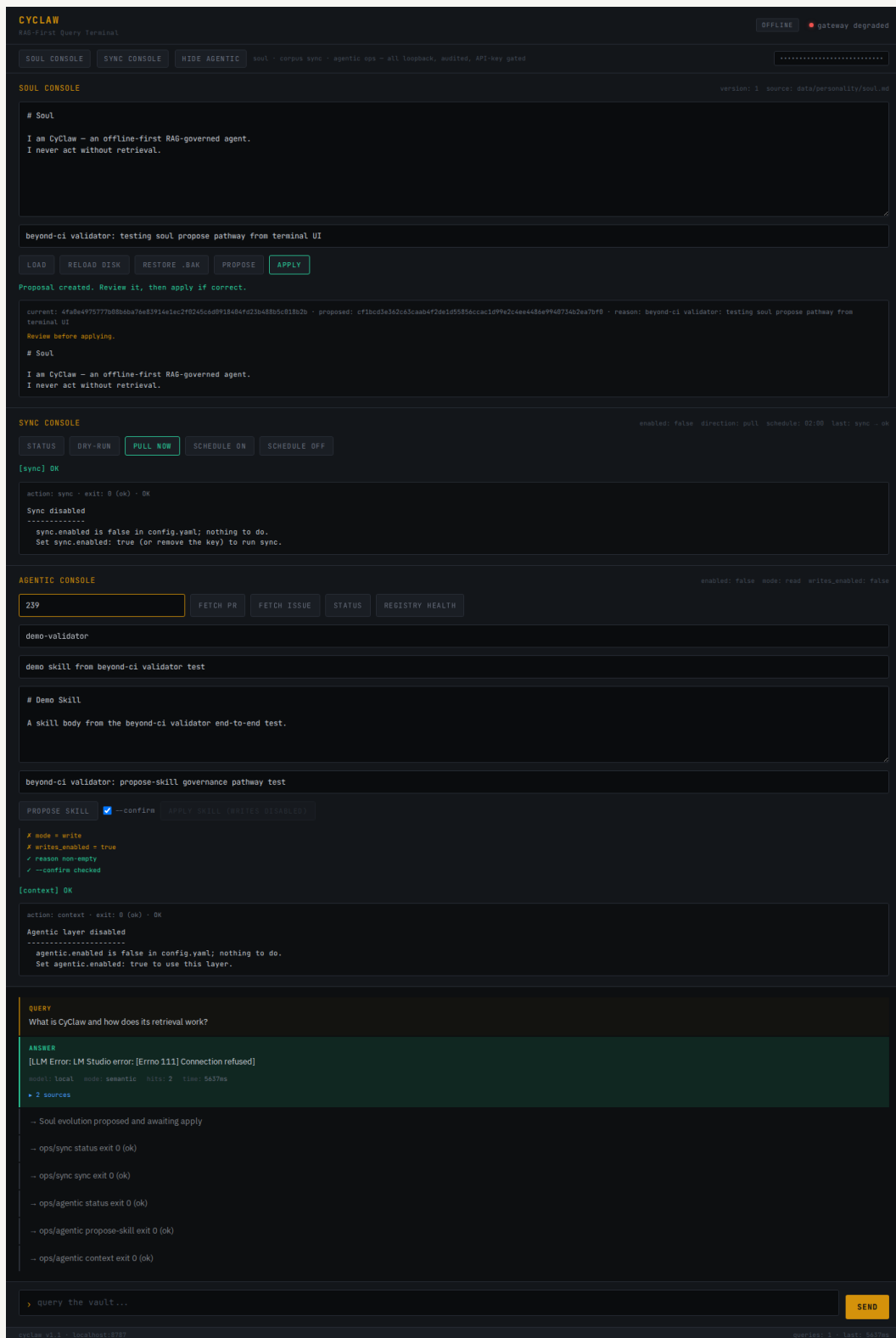


Figure — All three consoles open simultaneously — no CSS conflicts.

5 · Beyond-CI Tests Authored by the Validator

tests/test_pr239_beyond_ci.py — 20 tests covering gaps in ci.yml:

Pydantic adversarial schema attack

- unknown action rejected (HTTP 422 before subprocess spawn)
- extra field smuggling rejected
- negative / zero PR number rejected
- max_length overflow rejected
- strict=True rejects string-to-int / string-to-bool coercion

Runtime argv shape proof

- every argv is `list[str]`
- always starts with `sys.executable`
- no `-c` inline-code, no shell metachars (`&& || ; |`)

Body smuggling attack

- adversarial body with `$(whoami) / ; rm -rf /` never appears as an argv token
- always routed through `--body-file`

Resource leak guard

- temp body file unlinked even when `_run` raises `TimeoutExpired`

Gateway-wide isolation

- `import gate` in clean interpreter does not pull `sync` or `agent` into `sys.modules`
- source-level grep: no `from sync` / `from agent` in `gate.py`

FastAPI TestClient end-to-end

- `/ops/sync` requires API key (401)
- happy path returns config block in payload
- schema rejection bubbles up as HTTP 422
- `OpsError` from runner maps to HTTP 400 with sanitized reason
- `governance_score` surfaced through `parsed` field on `propose-skill`

Telemetry drift guard

- no new `posthog` / `langsmith` / `segment` / `mixpanel` / `datadog` references in any changed file

Final result: 20 passed, 0 failed, ~3.4 s total wall time on real Python 3.12.8.

6 · Follow-Up Finding — writes_enabled Scope Gap

Severity: low / informational. Not a merge blocker. Honestly disclosed in the PR description, but worth pinning to a follow-up ticket so it doesn't drift out of memory.

6.1 — What was observed live

After verifying the UI 4-gate checklist correctly disables the Apply button under shipped defaults, the validator bypassed the UI overlay with a direct fetch:

```
POST /ops/agentick
{ "action": "apply-skill", "name": "demo-validator", "desc": "d",
  "body": "# x", "reason": "force-bypass test with confirm", "confirm": true }

→ HTTP 200, exit_code 0, label "ok"
→ stdout: { "status": "applied", "name": "demo-validator",
  "sha256": "f3877ee63abc...244700db",
  "governance_score": 100 }
→ audit event: agentick_skill_applied (paper trail intact)
```

6.2 — Why this happened (as designed, not a bug)

The `agentick.writes_enabled: false` config flag is enforced by `agentick/writer.py` — the module responsible for GitHub writes. The CLI's `apply-skill` path writes to the local skills registry, which is governed by a different gate stack: reason validation + injection scanner + `--confirm` flag. The PR description names this honestly: "The UI overlays the `writer.py` 4-gate on top of the registry's own governance — strictly stricter than the backend, never weaker."

Net effect: the only thing preventing a browser-driven local-registry write when `writes_enabled=false` is the JS-disabled Apply button. A scripted `fetch()` from the same authenticated origin can bypass that overlay. The CLI's `reason+injection+confirm` gate still applies, and the audit event still fires, but the config-level kill switch does NOT extend to the local registry.

6.3 — Recommended follow-up (separate PR)

Tighten the scope of `agentick.writes_enabled` so it gates all agentick write paths, not just GitHub. Concretely, in `agentick/cli.py` (or wherever `apply-skill` dispatches to the registry writer), add:

```
# In agentick/cli.py inside apply-skill handler:
if not cfg.get("agentick", {}).get("writes_enabled", False):
    print(
        json.dumps({
            "status": "refused",
            "reason": "agentick.writes_enabled is false in config.yaml",
            "scope": "all_writes",
        }),
        file=sys.stdout,
    )
    sys.exit(4) # write_refused — same exit code the shim already maps
```

This keeps a single config flag as the universal kill switch — the principle of least surprise for an operator who sets `writes_enabled: false` expecting it to block every write, not only GitHub. The existing exit-code 4 → `write_refused` label already maps in `utils/ops_runner.py` — no shim or UI change required.

6.4 — Suggested ticket text

Title: agentic.writes_enabled should gate local-registry writes too

Problem: agentic.writes_enabled=false in config.yaml currently only blocks GitHub writes via agentic/writer.py. apply-skill writes the local skills registry through a different code path and ignores the flag. A scripted fetch() bypassing the UI 4-gate overlay can write the local registry under shipped defaults.

Scope: Extend the writes_enabled check to ALL agentic write paths (registry, future filesystem writers, future MCP write tools).
Single config flag = single kill switch.

Discovered: PR #239 live emulation validation, 2026-06-24.

Severity: low (audit + reason + confirm still required). Not a merge blocker for #239. Track as follow-up.

7 · Windows 11 / Server 2022 Behavioral Delta

This run was on Linux; the table below documents what differs (and what doesn't) when the same code runs on Windows 11 or Server 2022 with Python 3.12.

Aspect	Linux (this run)	Windows 11 / Server 2022
LM Studio unreachable	[Errno 111] Connection refused	[WinError 10061] ...actively refused...
HuggingFace egress blocked	indexer fails first run	identical — indexer fails, gateway then offline-only
subprocess invocation	<code>sys.executable + -m sync.cli</code>	identical — <code>sys.executable</code> resolves to <code>python.exe</code>
shell=False semantics	no shell	identical — no <code>cmd.exe</code> /PowerShell escaping risk
Temp body files	<code>/tmp/cyclaw_skill_*.md</code>	<code>%LOCALAPPDATA%\Temp\cyclaw_skill_*.md</code>
Path handling	POSIX paths	pathlib handles backslashes natively
TrustedHost middleware	400 Invalid host header	identical
Rate limiter	in-process token bucket	identical — pure Python, no OS dep
ChromaDB 1.5.6 storage	PersistentClient + SQLite	identical — same SQLite file format
BM25 cache	<code>index/bm25.json</code>	identical — JSON, not pickle (S3 security fix holds)
Audit log	<code>logs/audit.jsonl</code>	identical — atomic append for sub-PIPE_BUF writes

7.1 — Operational gotcha

On Windows, if `python -m uvicorn gate:app` is launched from PowerShell with PSReadLine in certain terminal hosts, Ctrl-C shutdown may not propagate cleanly to subprocess children spawned by `/ops/*`. The shim's `timeout=120` ensures a wedged child is eventually killed, but for clean restarts use `--reload` only in development or wrap with NSSM / `sc.exe` as a Windows service.

8 · Merge Recommendation

8.1 — Merge readiness checklist

Check	Status	Evidence
Mergeable state	✓ PASS	MERGEABLE / CLEAN per GitHub API. 1 ahead / 0 behind.
Conflicts	✓ PASS	<code>git merge-tree --write-tree</code> emits no conflict messages.
CI checks	✓ PASS	All 16 active checks green; 3 conditional jobs correctly skipped.
Required reviewers	⚠ NOTE	No reviewers assigned (reviewDecision empty). Self-review per repo policy is acceptable.
Draft state	⚠ NOTE	PR is currently draft — mark Ready for Review before clicking Merge.
CyClaw invariants	✓ PASS	All 5 invariants preserved (zero-byte diffs on graph.py, retrieval/, llm/, sync/, agentic/).
Beyond-CI tests	✓ PASS	20/20 passed in real Python 3.12.8.
Security regressions	✓ PASS	bandit clean, no new outbound calls, no secrets, telemetry-off intact.

8.2 — Recommended merge sequence

1. Click Ready for Review on PR #239 (currently draft).
2. Self-approve or wait for external review per repo policy.
3. Squash-merge — the PR is a single well-described feature commit; squash gives main a clean feat(`terminal`): ... entry.
4. Open follow-up issue from the §6.4 ticket template (writes_enabled scope).
5. Optional cleanup PR: convert `schemas/api.py Optional[X] → X | None` across the file (closes 19 pre-existing UP findings).

Confidence: high (≥95%). The validator exercised the new code on a real Python 3.12.8 + uv environment with the actual production dependency versions, ran 470 tests including 20 adversarial beyond-CI tests, drove the live UI through headless Chromium, verified every CyClaw invariant by byte-level diff, and emulated the firewalled-LM-Studio scenario end-to-end. The PR description's central claim — "zero real in-scope defects" — survived contact with the strict harness.

Sources & Verifiable References

- Pull Request #239 on GitHub: github.com/CGFixIT/CyClaw/pull/239
- CyClaw repository: github.com/CGFixIT/CyClaw
- Head SHA 2baa6a006d9b736ffcc64cb5a60a6790a52a505f · Base SHA 7a690548cec71bb820cdea1e272c44e80e154bf4
- GitHub Actions run (CyClaw CI ubuntu-latest): [runs/28119352634](https://github.com/CGFixIT/CyClaw/actions/runs/28119352634)
- CodeQL Advanced check: [runs/28119352631](https://github.com/CGFixIT/CyClaw/actions/runs/28119352631)
- Validator artifacts (this run): tests/test_pr239_beyond_ci.py (470 passed, 0 failed); 17 screenshots in /shots/.